

VAIJUNTO · SISTEMA DE CARONAS COMPARTILHADAS

Plano de Execução

Do servidor de eco ao sistema completo, em quatro dias

Disciplina Concorrência e Conectividade (PBL)

Linguagem Go 1.27

Meta funcional segunda-feira

Entrega final quinta-feira

1. Contexto

O PBL vence **quinta-feira**, e a meta é ter o sistema **funcional na segunda**. Isso permite separar duas coisas que não cabiam juntas: segunda é para fazer funcionar, terça em diante é para entender cada linha, provar a concorrência e empacotar.

Hoje existe o servidor de eco em `pbl/` (servidor e cliente TCP, uma goroutine por conexão), que já cobre **socket TCP e concorrência básica**. O que falta é o domínio: login, caronas, busca por trecho, reserva atômica multi-trecho, expiração de 10 minutos, persistência em JSON, testes e Docker.

As duas decisões que fazem o projeto caber no prazo

DECISÃO 1 — UM ÚNICO MUTEX GLOBAL

Um `sync.Mutex` protegendo todo o estado. Sem locks por carona, sem locks aninhados, sem risco de deadlock. O efeito colateral é enorme: a **atomicidade multi-trecho** — a parte mais difícil do enunciado — sai de graça, porque tudo acontece dentro de um único `Lock`.

DECISÃO 2 — UMA STRUCT DE PEDIDO SÓ

Um `Pedido` com todos os campos possíveis, em vez de um tipo por ação. Nada de interfaces ou `json.RawMessage`. Feio no papel, trivial de entender e de depurar no terminal.

2. Onde está a nota

Se algo tiver que ser cortado, corte de baixo para cima.

#	Item	Por quê
1	Reserva atômica + mutex	É o objetivo declarado da disciplina
2	Teste com <code>go test -race</code>	Prova objetiva de tratamento de concorrência
3	Socket TCP + goroutines	Já pronto no eco
4	Expiração de 10 minutos	Requisito explícito do enunciado
5	Login	Requisito explícito, mas trivial
6	Docker	Requisito explícito, e é só configuração
7	Persistência em JSON	Requisito, mas o menos arriscado de perder
8	Busca com conexão	Enriquece muito a demonstração
9	Cliente CLI bonito	Não vale nota nenhuma

3. Estrutura final

```
pbl/  
  go.mod                                module vaijunto  
  protocolo/protocolo.go                o que trafega na rede  
  dados/dados.go                        estado + mutex + regras  
  dados/dados_test.go                  o teste de corrida  
  servidor/main.go                     rede e roteamento  
  cliente/main.go                      menus de terminal  
  Dockerfile  
  docker-compose.yml
```

Quatro pacotes. `servidor` e `cliente` continuam sendo `package main`; `protocolo` e `dados` são bibliotecas.

4. Módulo protocolo

Resolve o problema de fronteira de mensagem do TCP: **uma linha = uma mensagem**, com o conteúdo em JSON.

pbl/protocolo/protocolo.go

```
type Pedido struct {
    Acao    string // "login", "cadastrar", "buscar", "reservar", "pagar"
    Usuario string
    Senha   string
    Rota    []string // cadastrar
    Data    string
    Assentos int
    Preco   int
    Origem  string // buscar
    Destino string
    Itens   []Item // reservar
    ReservaID int // pagar
}

type Item struct { CaronaID, De, Ate int } // De/Ate sao indices na rota
```

Função	Assinatura	O que faz
LerPedido	(*bufio.Reader) (Pedido, error)	Lê até <code>\n</code> , faz <code>json.Unmarshal</code>
LerResposta	(*bufio.Reader) (Resposta, error)	O mesmo, do lado do cliente
EnviarPedido	(io.Writer, Pedido) error	<code>json.Marshal</code> + <code>\n</code>
EnviarResposta	(io.Writer, Resposta) error	<code>json.Marshal</code> + <code>\n</code>

`Resposta` carrega `OK bool`, `Erro string`, `Caronas []Carona`, `Itinerarios [][]Item`, `ReservaID int` e `Mensagem string`.

5. Módulo dados — o coração

pbl/dados/dados.go

```

type Banco struct {
    mu      sync.Mutex
    Usuarios map[string]Usuario
    Caronas []Carona
    Reservas []Reserva
    proxID  int
}

type Carona struct {
    ID int; Motorista string
    Rota []string      // ["Feira", "Salvador", "Ilheus"]
    Data string
    Assentos int        // total por trecho
    Ocupados []int      // len = len(Rota)-1, um contador por trecho
}

type Reserva struct {
    ID int; Passageiro string
    Itens []protocolo.Item
    Estado string        // "pendente" | "paga" | "expirada"
    Expira time.Time
}

```

A IDEIA CENTRAL DA MODELAGEM

`Ocupados` é um contador por trecho. Reservar Feira→Ilhéus incrementa os trechos 0 e 1; reservar Salvador→Ilhéus incrementa só o trecho 1. **A disponibilidade por trecho sai naturalmente**, sem estrutura extra.

Funções públicas

Todas abrem com `mu.Lock()` e `defer mu.Unlock()`.

Função	Assinatura	Regra
<code>Autenticar</code>	<code>(login, senha string) (Usuario, bool)</code>	Compara com o mapa
<code>CadastrarCarona</code>	<code>(motorista string, rota []string, data string, assentos, preco int) int</code>	Cria <code>Ocupados</code> zerado com <code>len(rota)-1</code>
<code>Buscar</code>	<code>(origem, destino, data string) [][]Item</code>	Itinerários diretos e com 1 conexão
<code>Reservar</code>	<code>(passageiro string, itens []Item) (int, error)</code>	Atômica — ver abaixo
<code>Pagar</code>	<code>(reservaID int) error</code>	<code>pendente</code> → <code>paga</code> ; recusa expirada
<code>ExpirarVencidas</code>	<code>()</code>	Devolve assentos de reservas vencidas
<code>Salvar</code> / <code>Carregar</code>	<code>(caminho string) error</code>	JSON em arquivo

Reservar — a função que vale a nota

Duas fases dentro de **um único** `Lock`:

1. FASE DE VERIFICACAO -- nao altera nada
para cada item, para cada trecho de De ate Ate:
se Ocupados[t] >= Assentos -> devolve erro, NADA mudou
2. FASE DE APLICACAO -- so roda se a fase 1 passou inteira
para cada item, para cada trecho: Ocupados[t]++
3. Cria Reserva{Estado: "pendente", Expira: agora + 10min}

Verificar tudo antes de aplicar qualquer coisa é o que garante o “*pegou um trecho não fica sem o outro*”. Como está tudo sob o mesmo mutex, nenhuma outra goroutine consegue se intrometer entre a fase 1 e a fase 2.

Funções internas

Sem lock — são chamadas de dentro das públicas.

Função	Para quê
<code>(c Carona) Indice(cidade string) int</code>	Posição na rota, ou -1
<code>(c Carona) TemVaga(de, ate int) bool</code>	Todos os trechos da faixa têm assento
<code>(c *Carona) Ocupar(de, ate int)</code>	Incrementa a faixa. Ponteiro: modifica
<code>(c *Carona) Liberar(de, ate int)</code>	Decrementa. Usada na expiração

REGRA DE OURO

Função pública trava, função interna **nunca** trava. Se uma função com lock chamar outra com lock, o programa congela. É o deadlock mais comum de quem está começando.

6. Módulo servidor

Mantém a estrutura do eco. Só muda o que acontece dentro de `atender`.

Função	O que faz
<code>main</code>	<code>Carregar</code> o JSON, subir a goroutine de expiração, <code>Listen(":8080")</code> , laço de <code>Accept</code>
<code>atender(conn, banco)</code>	Laço: <code>LerPedido</code> → <code>executar</code> → <code>EnviarResposta</code>
<code>executar(p, *sessao, banco)</code>	<code>switch p.Acao</code> — o roteador do sistema
<code>expirarPeriodicamente(banco)</code>	<code>time.Ticker</code> de 30s chamando <code>ExpirarVencidas</code>

POR QUE A SESSÃO NÃO PRECISA DE MUTEX

`sessao` guarda quem logou naquela conexão. Como ela é uma variável local de `atender`, **cada goroutine tem a sua** e ninguém compartilha nada. Estado local não precisa de proteção — só o que é compartilhado precisa.

Toda ação que não seja `login` verifica `sessao.Usuario != ""` antes de executar.

7. Módulo cliente

Função	O que faz
<code>main</code>	Conecta, pede login, direciona para o menu do tipo de usuário
<code>pedir(p Pedido) Resposta</code>	Envia e espera a resposta. Usada por todas as outras
<code>menuMotorista</code>	Cadastrar carona, listar as minhas
<code>menuPassageiro</code>	Buscar, escolher itinerário, reservar, pagar

Menu numerado lido do teclado. Sem firula — a interface não vale nota.

8. Cronograma

SEGUNDA — deixar funcional (~8h)

Fase	Tempo	Entrega verificável
1. <code>go.mod</code> + <code>protocolo</code>	1h	Cliente manda JSON, servidor devolve JSON
2. <code>dados</code> + <code>Autenticar</code> + login	1h30	Login funciona, senha errada é recusada
3. <code>CadastrarCarona</code> + <code>Buscar</code>	2h	Motorista cadastra, passageiro encontra
4. <code>Reservar</code> atômica	2h	Reserva de 2 trechos, tudo ou nada
5. Expiração de 10 minutos	1h	Assento volta sozinho
6. Cliente com menus	1h30	Fluxo completo pelo terminal

Fim de segunda: o sistema roda de ponta a ponta na sua máquina.

TERÇA — entender e provar

Fase	Tempo	Entrega
7. Revisão linha por linha	3h	Explicação de tudo que foi escrito
8. Teste com <code>-race</code>	1h30	50 goroutines no último assento; só 1 vence

QUARTA — empacotar

Fase	Tempo	Entrega
9. Docker	1h	<code>docker compose up</code> sobe tudo
10. Persistência JSON	45min	Reinicia sem perder dados
11. Busca com conexão	45min	Itinerário de 2 caronas

QUINTA — entregar

README, roteiro de demonstração e revisão final. Folga proposital para o que atrasar.

O teste da fase 8

```
carona com 1 assento
50 goroutines chamando Reservar ao mesmo tempo
sync.WaitGroup espera todas terminarem
verifica: exatamente 1 sucesso, 49 erros, Ocupados[0] == 1
```

A DEMONSTRAÇÃO DA APRESENTAÇÃO

Rode `go test -race ./...` e veja passar. Depois **comente o** `mu.Lock()` e rode de novo: o detector acusa a corrida e a contagem sai errada. Esses dois resultados lado a lado são a prova objetiva de que você tratou concorrência.

9. Detalhes que economizam tempo

- `":8080"` e não `"localhost:8080"` no servidor. Com `localhost` ele recusa conexões de fora da máquina e quebra no Docker.
- **Senha em texto puro** no JSON. É PBL, não produção. Não gaste tempo com hash.
- **Salvar em disco a cada alteração**, não periodicamente. Mais simples e evita mais um problema de concorrência.
- **Data como string** no formato `"2026-09-14"`. Comparação exata, sem `time.Parse`.
- **Reaproveite o eco**. O `bufio`, o `defer`, o laço de `Accept` e a goroutine por cliente continuam idênticos.

10. Como trabalhar

Segunda — ritmo de construção

Cada fase é escrita inteira, com comentário curto explicando o essencial na hora. Rodar, conferir que funciona, seguir. Sem parar para entender cada linha: o objetivo do dia é ver o sistema de pé.

Terça — ritmo de estudo

Voltar ao código já pronto e funcionando, e então explicar linha por linha. É aí que as dúvidas entram, e o que estiver confuso é reescrito.

POR QUE NESSA ORDEM

Estudar código que **já funciona** é mais fácil do que estudar código que você ainda não viu rodar: você já sabe o que ele faz, e só precisa entender como. A ordem inversa obriga a segurar dois problemas na cabeça ao mesmo tempo.

11. Verificação final

```
cd pbl
go vet ./...
go test -race ./...          # o teste de corrida passa
docker compose up --build    # servidor sobe
docker compose run cliente   # conecta e opera
```

Roteiro de demonstração

1. Motorista cadastra uma carona com **1 assento**
2. Dois passageiros tentam reservar o mesmo assento ao mesmo tempo

3. Apenas um consegue; o outro recebe erro
4. Quem conseguiu **não paga**
5. Passados 10 minutos, o assento volta a ficar livre sozinho
6. O segundo passageiro agora consegue reservar